另一種設計提案

保留建置的效率，
走向受控的正式部署。

Windows 正式部署治理計劃

GitLab 建置 → Azure DevOps Server 管理放行

正式換版，由授權部門掌握。



Express 作為 POC 起點

目標 12/31 · 待共同確認

六個問題，一條清楚的導入路線

先說明預期價值，再區分需求、規劃與待驗證項目



先看價值與邊界



再看設計與分工



最後走向落地



01 預期效益

02 正式主機存取邊界

03 需求狀態與目標架構

04 分支與部署策略

05 授權換版與復原設計

06 POC 到正式上線

讓換版不再每次重新摸索

完成導入並通過驗收後，預期帶來三項改善

01

維運更一致

一致流程，少搬檔、少漏步。

人員、時間、主機與版本
可追查。

02

降低操作風險

版本與目標綁定放行。

備份與復原演練，
降低失敗影響。

03

複用到其他專案

留下治理與操作範本。

各專案的服務與復原方式，
仍需個別驗證。

預期改善，不是既有成果；POC / 試辦後再以紀錄比較。

只送檔，也已越過正式主機存取邊界

原方案已否決：建置端不能直接碰正式主機



保留建置自動化；不以共用目錄、帳號或中繼腳本繞過邊界。

先把「需求」與「做得到」分開

官方文件支持工具能力，不代表本案已完成整合或驗收

需求已確認

已確認的需求

保留 GitLab 建置；正式主機僅限指定部門存取。

正式部署須先選定版本與目標、授權放行；送檔也算存取。

做法規劃中

規劃中的做法

以 Azure DevOps Server 管理放行，由目標 Agent 執行受控腳本。

納入備份、健康檢查與復原；需另行實作，並非工具自帶完整機制。

POC 待驗證

POC 待驗證

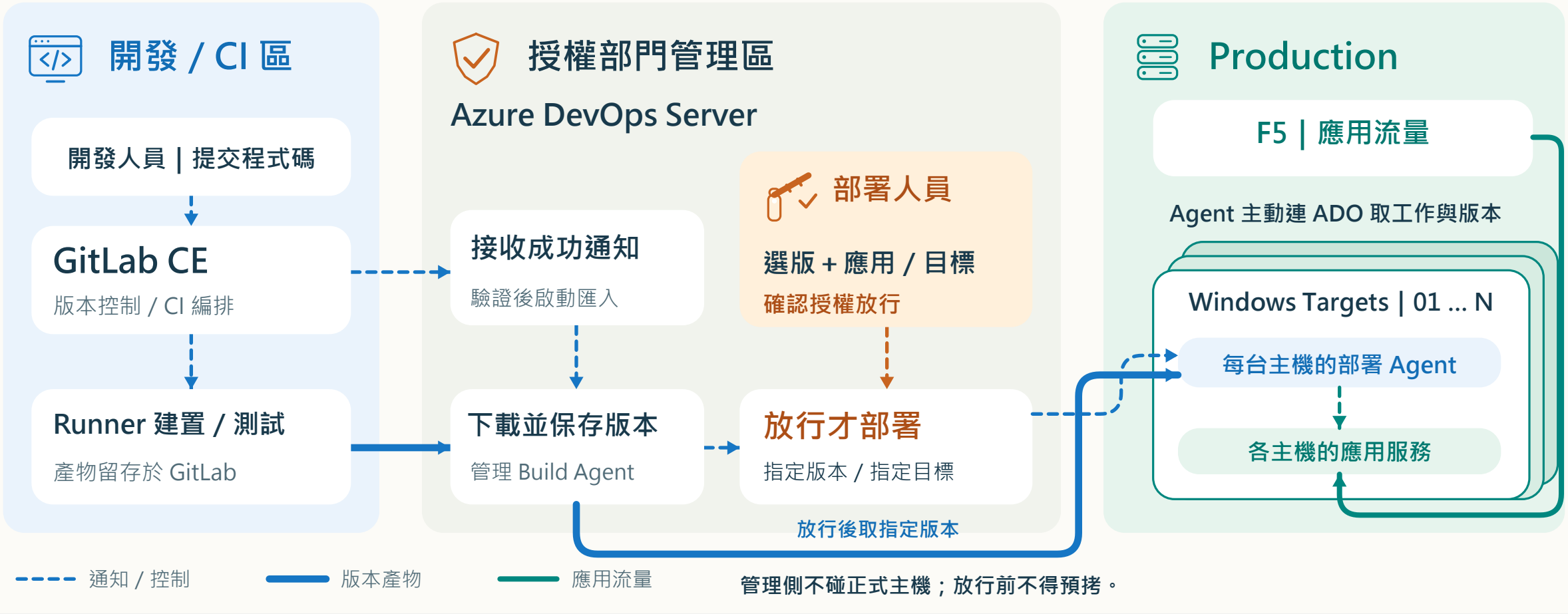
GitLab 串接、Agent 權限與放行前禁止送檔。

F5 隔離、人工確認、備份還原及失敗停止；含失聯例外。

從程式碼出發，在放行後抵達正式環境

端到端目標架構 / 待 POC：建置、版本管理與正式部署，各自守住責任邊界。

本頁看交付與權限分工；單機換版、人工功能確認與復原流程 → 第 8 頁。



不同 branch，各走合適的部署路線

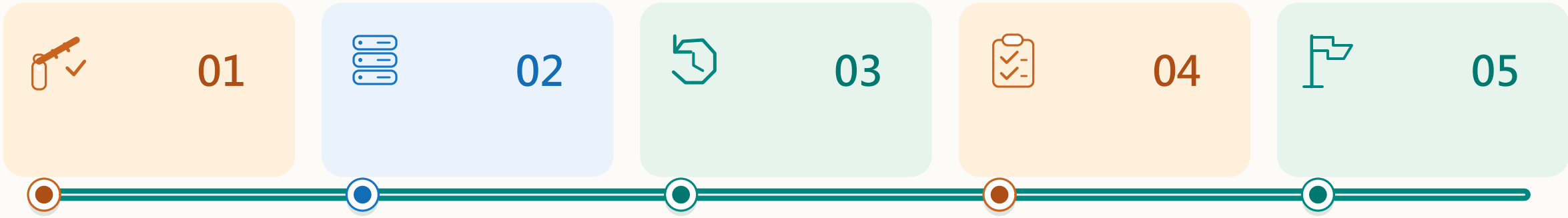
規劃示意 / 待 POC：各分支各自建置；正式部署仍須授權。

來源分支	觸發與策略	執行者	目標環境
master	符合條件自動觸發	Azure DevOps Server + Agent	SIT
pre-production	符合條件自動觸發	Azure DevOps Server + Agent	UAT
production / production-bq1 production-bq2 / production-nh …（平行正式分支示例）	選版 + 選目標 授權放行	Azure DevOps Server + Agent 依目標採不同換版策略	Production

不同分支可有不同 pipeline；不將 production 前綴自動視為已授權。

放行後才送檔，流量恢復前再確認

規劃 / 待 POC：Agent、備份、人工等待與復原腳本均須實作及演練。



保持隔離 → 還原舊版 → 再檢查

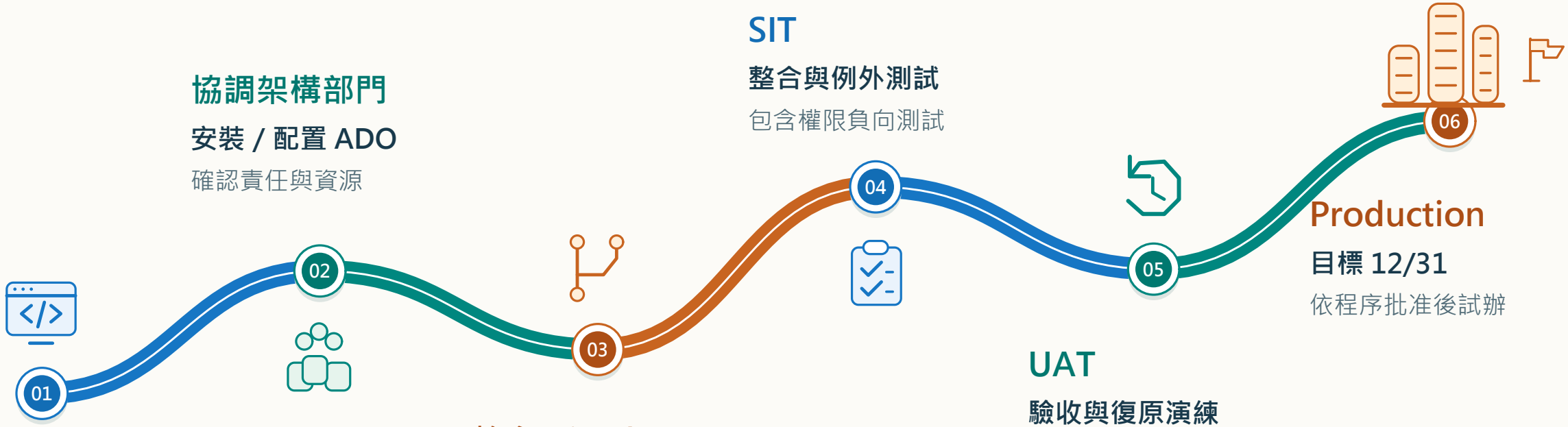
舊版健康且人工確認，才可上 F5。
復原成功仍記為「換版失敗」。

未確認、逾時或復原失敗：不恢復流量

停止後續目標並通知；主機失聯另需復原手段。

從第一個 POC，走向受控的正式上線

暫定里程碑順序；階段日期、資源與上線批准，待相關部門共同確認。



POC

驗證通知與權限

隔離評估環境

協調架構部門

安裝 / 配置 ADO

確認責任與資源

整合 GitLab

串接通知與產物

保留與稽核

SIT

整合與例外測試

包含權限負向測試

UAT

驗收與復原演練

由授權人員驗收

Production

目標 12/31

依程序批准後試辦

逐關通過再上線：先證明「未放行不能送檔」，再完成驗收與復原演練。